AI Engineer Roadmap

Oct 20, 2026 to Jan 11, 2027 · 12 weeks · 12 hours a week · start applying about Jan 12 · updated Oct 23

The goal

Be interview-ready for **LLM application engineer** roles in India and remote, and start applying about **Jan 12**. Keep applying through March. Hard limit: start applying by the **end of February**, even if the project is not perfect.

The weekly split (12 hours)

Track	Weeks 1 to 6	Weeks 7 to 12
AI project	6 h	6 h
DSA in Python	2.5 h	2.5 h
Design (LLD, then HLD)	2 h	3.5 h
SQL ladder	1.5 h	done

Half AI, half fundamentals. The SQL hours move to design once the ladder is done.

Where you are now (Oct 23)

Done

- Python exercises, Levels 1 to 4
- `analyze.py` reads the real CSV: `parse_amount` works, totals are correct
- AI term map written (`ai_term_map.md`)
- Fundamentals: OOP, SQL, your first ER and class diagrams, the 7-step design method

Still open in week 1

- **Level 5:** the asserts still fail. Four bugs: `__init__` is empty, `=+` instead of `+=`, `withdraw` doesn't subtract, and `__repr__` has no brackets.
- **Learn the term map:** explain each term out loud in one sentence.
- **Sat Oct 24:** design practice: a cab booking case. Afterwards, write down every question and how you answered it.

The rules

> **1. You type every line of practice code.** AI explains, gives hints and reviews. Ask for the full answer ("show me") only after 10 minutes stuck, and then retype it yourself.

> **2. Say it out loud.** After each topic, explain it in one minute without notes. If you can't, you haven't learned it yet.

> **3. Every Sunday, review the week.** Tick it, and write three lines: what worked, what didn't, what's next.

> **4. Build small, then grow.** Get one feature working end to end before adding the next.

Week by week

✓	Wk	Dates	AI project (6 h)	Fundamentals (6 h)
☐	1	Oct 20–26	`parse_amount` and `analyze.py` (done). Term map (written, learn it). Finish Level 5.	Fundamentals: OOP, SQL exercises, ER and class diagrams. Cab booking design case on Saturday.
☐	2	Oct 27–Nov 2	Make a virtual environment. Build a FastAPI hello endpoint, then a POST endpoint with a Pydantic `Transaction`. Open `/docs`. Then a CSV upload endpoint that returns your `analyze.py` results as JSON. Push the repo to GitHub.	**DSA:** re-solve 5 easy C# problems in Python (arrays, strings, hash maps). **Design:** the library case as a console app, typed by you. Lists in the service first, then move them into a repository. **SQL:** ladder Levels 0 and 1 (SQLBolt 1–6, SQL 50 "Select").
☐	3	Nov 3–9	First Claude call: ask for structured JSON that categorises each transaction. Keep the API key out of Git. Write your first `pytest` tests.	**DSA:** 5 more easy problems, then two pointers. **Design:** the loan tracker case end to end, plus one change request. **SQL:** Level 2, aggregation.
☐	4	Nov 10–16	Hand-label 30 to 50 transactions and measure accuracy. Add retries and async parallel calls. Checkpoint 1 The service runs locally and you have an accuracy number.	**DSA:** sliding window and stack. **Design:** parking lot (a classic LLD case). SOLID with your own examples. **SQL:** Level 3, joins.
☐	5	Nov 17–23	Learn embeddings and chunking. Load public loan-policy documents into Chroma or pgvector.	**DSA:** linked lists and binary search. **Design:** HLD building blocks, one page each: load balancer, cache, queue, replicas, sharding. **SQL:** Level 4, subqueries and CTEs.
☐	6	Nov 24–30	Build RAG by hand with the plain SDK: retrieve, then answer with citations. Test whether the right chunk comes back. Checkpoint 2	**DSA:** trees (traversals, BFS and DFS). **Design:** HLD 1, a URL shortener, using the 6 steps. **SQL:** Level 5, window functions. Take the HackerRank SQL Intermediate test.
☐	7	Dec 1–7	Rebuild the same RAG in LangChain, and write down what it hides. Add tool calling so the model can call your analysis functions.	**DSA:** heaps and top K. **Design:** HLD 2, a notification system and a rate limiter.
☐	8	Dec 8–14	Build an agent loop that answers "Is this applicant affordable?" using the statement tools and the policy documents.	**DSA:** graphs, BFS and DFS. **Design:** HLD 3, a news feed. Read Chip Huyen, chapter by chapter.

☐	9	Dec 15–21	Learn MCP and build one small server that exposes the analysis. Put the service in Docker.	**DSA:** dynamic programming basics (climbing stairs, house robber, coin change). **Design:** LLM system design 1, a RAG pipeline.
☐	10	Dec 22–28	Deploy to Azure Container Apps with secrets, logging and a health check. Checkpoint 3 A public URL works.	**DSA:** more DP, and intervals. **Design:** LLM system design 2, an LLM gateway (rate limits, retries, caching, cost).
☐	11	Dec 29–Jan 4	Write the README: architecture diagram, eval numbers, trade-offs. Rewrite your resume around the project and the work AI feature.	**DSA:** mixed review, two timed sets. **Design:** draw your own project's architecture and explain every trade-off out loud.
☐	12	Jan 5–11	Mock interviews: Python coding, SQL, "design a RAG pipeline", and explain every term on the term map.	**DSA:** timed mocks. **Design:** three mock rounds: one LLD, one HLD, one RAG. **Start applying about Jan 12.**

Side track: the AI feature at work

- **Weeks 2 and 3:** read the AI assessment code. Understand how it loads the data, builds the prompt and calls Claude.
- **Week 4 onwards:** own one small improvement. It's the strongest line on your resume.
- Use the company API key only for company work. The personal project uses your own key.

If you fall behind

Never cut

- FastAPI with Pydantic
- The Claude accuracy test (evals)
- RAG built by hand
- The Azure deploy
- At least 2 hours of DSA a week

Cut first

- The LangChain rebuild shrinks to one day.
- MCP becomes reading only.
- HLD 3 (news feed) becomes reading only.

Checkpoints in weeks 4, 6 and 10 tell you early if you're behind.